

Regras e Orientações para Entrega:

1. **Formato:** Relatório em PDF contendo os códigos, comentários, prints de execução e respostas aos questionários quando necessário.
2. **Linguagens Permitidas:** Os trabalhos podem ser feitos em **GNU Octave**, **Python** ou **C/C++**.
3. **Composição dos Grupos:** Podem ser feitos **individualmente** ou em **dupla**.

1. Objetivos da Aula

Nesta prática de laboratório, você implementará algoritmos computacionais fundamentais de contagem e geração combinatória, conectando a teoria matemática clássica a técnicas algorítmicas (recursão, decomposição e busca combinatória). Os objetivos principais são:

- Implementar rotinas para cálculo analítico de contagens (Fatorial, Combinações Simples, Permutações com Repetição, Permutações Circulares e Arranjos).
- Desenvolver funções recursivas e iterativas para geração explícita de conjuntos de permutações, subconjuntos combinatórios e o conjunto das partes (*Power Set*).
- Aplicar modelos combinatórios na resolução de problemas reais de computação: análise do espaço amostral de senhas, composição de comitês sob restrições e verificação de anagramas.

2. Fundamentação Teórica e Atividades Práticas

2.1 Fatorial e Combinações Simples

O **Fatorial** de um número natural n ($n!$) é o produto de todos os inteiros positivos de 1 até n , com a convenção de que $0! = 1$. Chamamos de **Combinação Simples** a cada um dos subconjuntos de k elementos distintos escolhidos a partir de um conjunto com n elementos, em que a **ordem dos elementos não importa**:

Definição Formal (Combinação Simples):

$$C(n, k) = n! / (k! \cdot (n - k)!)$$

Em problemas práticos com restrições (como a proibição de elementos coexistirem), calculamos o total irrestrito e subtraímos a quantidade de agrupamentos indesejados.

EXERCÍCIO 1 Implementando a Fórmula de Combinação

Crie uma função `combinacao(n, k)` que calcule a combinação de n elementos tomados k a k .

A sua função deve:

1. Receber dois números inteiros, n e k .
2. Verificar se n é maior ou igual a k . Se não for, retorne uma mensagem de erro.
3. Utilizar uma função auxiliar para o cálculo do fatorial (ou implementá-lo diretamente).
4. Aplicar a fórmula: $C(n, k) = n! / (k! \cdot (n - k)!)$.
5. Retornar o resultado.

Exemplo: `combinacao(5, 2)` deve retornar `10`.

EXERCÍCIO 2 O Problema do Comitê

Em uma empresa com 10 funcionários, 4 precisam ser selecionados para um comitê.

1. Escreva um programa que use a função de combinação do Exercício 1 para calcular quantas maneiras diferentes de formar o comitê existem.
2. Agora, imagine que, dentre os 10 funcionários, 2 não se dão bem e não podem estar no mesmo comitê. Modifique seu programa para calcular quantas combinações de comitês são possíveis sem que esses dois funcionários estejam juntos.

Dica: Você pode calcular o total de comitês possíveis, e depois subtrair o número de comitês que contêm os dois funcionários incompatíveis.

EXERCÍCIO 3 Analisando o Jogo da Loteria

Um jogo de loteria pede para você escolher 6 números diferentes de um total de 60.

1. Usando sua função de combinação do Exercício 1, calcule o número total de combinações possíveis (o número de bilhetes diferentes que você pode jogar).
2. Qual a probabilidade de acertar os 6 números, considerando que você jogou apenas um bilhete? A probabilidade é 1 dividido pelo número total de combinações.

2.2 Permutações Simples e Algoritmos de Geração

A **Permutação Simples** de n elementos distintos é qualquer agrupamento ordenado contendo todos os elementos do conjunto ($P_n = n!$). Para **gerar** computacionalmente as permutações, utiliza-se estratégia recursiva: para cada elemento, fixa-se ele como o primeiro da sequência e concatenam-se as permutações recursivas dos elementos restantes.

EXERCÍCIO 4 Gerando Permutações de uma Lista

Diferente do exercício anterior, este desafio é sobre gerar as permutações em si, não apenas contá-las.

1. Crie uma função chamada `gerar_permutacoes(lista)` que recebe uma lista de elementos únicos.
2. A função deve retornar uma lista de todas as permutações possíveis dos elementos da lista de entrada.
3. Você pode usar recursão para resolver este problema. A lógica é: para cada elemento na lista, fixe-o como o primeiro elemento e encontre todas as permutações dos elementos restantes.

Exemplo: `gerar_permutacoes(['A', 'B', 'C'])` deve retornar uma lista com todas as 6 permutações, como `[['A', 'B', 'C'], ['A', 'C', 'B'], ...]`.

Dica: O módulo `itertools` do Python tem uma função chamada `permutations` que faz isso. Tente implementar a sua própria versão antes de olhar para a documentação!

EXERCÍCIO 5 Anagramas com Python / Linguagem Escolhida

Um anagrama é uma palavra formada pela transposição das letras de outra palavra.

1. Crie uma função `encontrar_anagramas(palavra)` que recebe uma string.
2. Use a função de geração de permutações do Exercício 2 para encontrar todas as permutações das letras da palavra.
3. Filtre a lista resultante para incluir apenas as permutações que são palavras válidas (você pode criar uma pequena lista de palavras válidas para teste).
4. Retorne a lista de anagramas encontrados.

Exemplo: `encontrar_anagramas('roma')` pode retornar `['amor', 'ramo', 'mora']` se estas palavras estiverem na sua lista de teste.

2.3 Permutações com Repetição e Permutações Circulares

Quando determinados elementos aparecem repetidos em uma coleção, trocá-los entre si não altera a ordem percebida. Da mesma forma, em arranjos cíclicos ou de mesa redonda, deslocamentos puramente rotacionais configuram a mesma permutação relativa.

Fórmulas Teóricas:

- **Permutação com Repetição:** $P_n^{(n_1, n_2, \dots, n_k)} = n! / (n_1! \cdot n_2! \cdot \dots \cdot n_k!)$
- **Permutação Circular:** $PC(n) = (n - 1)!$

EXERCÍCIO 6 Permutações com Repetição

Os exercícios anteriores lidavam com elementos únicos. Agora, vamos lidar com a repetição.

1. Crie uma função `perm_com_repeticao(string)` que recebe uma string (que pode conter caracteres repetidos) e calcula o número de permutações únicas.
2. A fórmula para isso é:

$$P(n) = n! / (n_1! \cdot n_2! \cdot \dots \cdot n_k!)$$

onde n é o comprimento total da string, e $n_1, n_2, \dots$ são as contagens de cada caractere repetido.

Exemplo: `perm_com_repeticao('AAB')` deve retornar `3`, que são as permutações: AAB, ABA, BAA.

EXERCÍCIO 7 Permutações Circulares

A Matemática Discreta também aborda a ordenação de objetos em um círculo.

1. Crie uma função `permutacao_circular(n)` que calcule o número de permutações de n elementos em um círculo.
2. A fórmula para isso é: $(n - 1)!$.
3. Explique por que a fórmula é diferente da permutação linear. (Dica: em um círculo, não há "início" ou "fim" definidos).

2.4 Geração de Combinações e Conjunto das Partes (Power Set)

Além da contagem, gerar os subconjuntos de tamanho k exige selecionar elementos preservando a unicidade dos subconjuntos (onde $\{A, B\} = \{B, A\}$). O **Conjunto das Partes** ou **Power Set** de um conjunto com n elementos consiste na coleção de todos os seus subconjuntos possíveis (de tamanho 0 até n), contendo exatamente 2^n subconjuntos.

EXERCÍCIO 8 Gerando Combinações de uma Lista

Assim como no Exercício 2, o desafio é gerar as combinações, não apenas contá-las.

1. Crie uma função `gerar_combinacoes(lista, k)` que recebe uma lista de elementos e um número inteiro k .
2. A função deve retornar uma lista de todas as combinações (subconjuntos) de tamanho k .
3. A ordem dos elementos dentro de cada subconjunto não importa.

Exemplo: `gerar_combinacoes(['A', 'B', 'C', 'D'], 2)` deve retornar `[['A', 'B'], ['A', 'C'], ['A', 'D'], ['B', 'C'], ['B', 'D'], ['C', 'D']]`.

EXERCÍCIO 9 Combinações de Subconjuntos (Power Set)

O conjunto de todas as combinações possíveis de um conjunto original é chamado de "Power Set" ou "Conjunto das Partes".

1. Crie uma função `power_set(lista)` que receba uma lista de elementos.
2. A função deve retornar uma lista de listas, contendo todas as combinações possíveis, de 0 elementos até n elementos.
3. Você pode usar a função de geração de combinações do Exercício 5 em um loop, variando k de 0 a `len(lista)`.

Exemplo: `power_set([1, 2, 3])` deve retornar `[[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]`.

2.5 Arranjos Simples e Modelagem de Espaços de Busca de Senhas

Diferente das combinações, no **Arranjo** a ordem em que os elementos são agrupados é essencial (por exemplo, na definição de senhas ou códigos). Sem repetição de caracteres, dispomos de $A(n, p) = n! / (n - p)!$ sequências. Com repetição livre, cada posição admite qualquer uma das n alternativas (n^p possibilidades).

EXERCÍCIO 10 O Problema das Senhas

Uma senha de 8 caracteres deve ser criada usando letras minúsculas (26) e dígitos (10), totalizando 36 opções disponíveis.

1. Calcule quantas senhas diferentes são possíveis se a repetição é permitida.
2. Calcule quantas senhas diferentes são possíveis se a repetição não é permitida. Use sua função de arranjo/fatorial para isso.
3. Qual dos dois cenários oferece um número maior de senhas? Por que a diferença é tão grande?

3. Critérios de Avaliação do Relatório

Critério	Descrição	Peso
Corretude Algorítmica	Implementação funcional das fórmulas e rotinas recursivas/construtivas solicitadas.	40%
Testes e Execução	Inclusão de prints de execução comprovando o correto funcionamento dos casos de teste.	30%
Rigor Teórico	Respostas detalhadas e tecnicamente embasadas para as questões dos exercícios 3, 6, 7 e 10.	20%
Organização e Documentação	Clareza na estrutura do relatório em PDF, comentários explicativos e boas práticas.	10%